

SKILL WEEK 4 OSSP

NAME:VARSHITH COMPLE

TASK 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

typedef struct
{
    char *tokens[MAX_TOKENS];
    int count;
} TokenList;

void tokenize(char *input, TokenList *list)
{
    list->count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && list->count < MAX_TOKENS - 1)
    {
        list->tokens[list->count++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    list->tokens[list->count] = NULL;
}

int validateTokens(TokenList *list)
{
    if (list->count == 0)
    {
        printf("Error: Empty command\\n");
        return 0;
    }

    for (int i = 0; i < list->count; i++)
    {
        if (strcmp(list->tokens[i], "|") == 0 ||
            strcmp(list->tokens[i], ">") == 0)
        {
            if (i == 0 || i == list->count - 1)
            {
                printf("Syntax Error: Invalid delimiter position\\n");
                return 0;
            }

            if (strcmp(list->tokens[i - 1], "|") == 0 ||
                strcmp(list->tokens[i - 1], ">") == 0)
            {
                printf("Syntax Error: Consecutive delimiters\\n");
                return 0;
            }
        }
    }

    return 1;
}

void debugTokens(TokenList *list)
{
    printf("\\n\\nTokens:\\n");

    for (int i = 0; i < list->count; i++)
    {
        printf("Token[%d] = %s\\n", i, list->tokens[i]);
    }
}
```

```
void executeCommand(TokenList *list)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child: Executing command...\n");
        printf("Child PID: %d\n", getpid());

        execvp(list->tokens[0], list->tokens);

        perror("execvp failed");
        exit(1);
    }
    else
    {
        printf("Parent: Waiting for child...\n");

        waitpid(pid, NULL, 0);

        printf("Parent: Child completed\n");
    }
}

int main()
{
    char input[MAX_INPUT];
```

```

while (1)
{
    printf("\nmyShell> ");
    fflush(stdout);

    int n = read(STDIN_FILENO, input, MAX_INPUT - 1);

    if (n <= 0)
        break;

    input[n] = '\0';

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command. Please enter a command.\n");
        continue;
    }

    if (strcmp(input, "exit") == 0)
        break;

    TokenList list;

    tokenize(input, &list);

    debugTokens(&list);

    if (!validateTokens(&list))
        continue;

    executeCommand(&list);
}

```

```

}

printf("\nShell terminated.\n");

return 0;
}

```

OUTPUT:

```

varshithcomple@VARSHITHs-MacBook-Air ~ % nano parser.c
varshithcomple@VARSHITHs-MacBook-Air ~ % gcc parser.c -o parser
clang: warning: overriding deployment version from '16.0' to '26.0' [-Woverriding-deployment-version]
varshithcomple@VARSHITHs-MacBook-Air ~ % ./parser

myShell> ls
Tokens:
Token[0] = ls
Parent: Waiting for child...
Child: Executing command...
Child PID: 89322
Applications  Downloads  File1.txt  first.c   is         Movies    Music      notes.txt  OSSP      Pictures  shell.c    t2.c       task1.c.save  tree
Desktop      exec       File2.txt  fork.c   is.c       ms        notes      op         parser    Public    t1        task1      task2         tree.c
Documents    exec.c    first     fork.c   Library   ms.c     notes.c    op.c       parser.c  shell    t1.c     task1.c    task2.c
Parent: Child completed

myShell> nano parser.c
Tokens:
Token[0] = nano
Token[1] = parser.c
Parent: Waiting for child...
Child: Executing command...
Child PID: 89428
Parent: Child completed

```

TASK 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

typedef struct
{
    char *args[MAX_ARGS];
    int count;
} Command;

int parseCommand(char *input, Command *cmd)
{
    int i = 0;
    int arg = 0;

    while (input[i] != '\0')
    {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        char *buffer = malloc(MAX_INPUT);

        if (buffer == NULL)
        {
            perror("malloc failed");
            return 0;
        }

        int j = 0;

        if (input[i] == '\\')
        {
            i++;

            while (input[i] != '\0' && input[i] != '\\')
            {
                buffer[j++] = input[i++];
            }

            if (input[i] != '\\')
            {
                printf("\nSyntax Error: Unmatched single quote.\n");
                free(buffer);
                return 0;
            }

            i++;
        }
        else
        {
            while (input[i] != '\0' &&
                    input[i] != ' ' &&
                    input[i] != '\t')
            {
                if (input[i] == '\\')
                {
                    i++;

                    while (input[i] != '\0' && input[i] != '\\')
                        buffer[j++] = input[i++];

                    if (input[i] != '\\')

```

```

        if (input[i] == '\\')
        {
            i++;

            while (input[i] != '\\0' && input[i] != '\\')
            {
                buffer[j++] = input[i++];
            }

            if (input[i] != '\\')
            {
                printf("\nSyntax Error: Unmatched single quote.\n");
                free(buffer);
                return 0;
            }

            i++;
        }
        else
        {
            while (input[i] != '\\0' &&
                    input[i] != ' ' &&
                    input[i] != '\\t')
            {
                if (input[i] == '\\')
                {
                    i++;

                    while (input[i] != '\\0' && input[i] != '\\')
                    {
                        buffer[j++] = input[i++];
                    }

                    if (input[i] != '\\')
                    {
                        printf("\nSyntax Error: Unmatched single quote.\n");
                        free(buffer);
                        return 0;
                    }

                    i++;
                }
                else
                {
                    buffer[j++] = input[i++];
                }
            }

            buffer[j] = '\\0';

            cmd->args[arg++] = buffer;

            if (arg >= MAX_ARGS - 1)
                break;
        }

        cmd->args[arg] = NULL;
        cmd->count = arg;

        return 1;
    }

void displayParsing(Command *cmd)
{
    printf("\nParsing Result:\n");

    for (int i = 0; i < cmd->count; i++)
    {

```

```

        for (int i = 0; i < cmd->count; i++)
        {
            printf("Argument %d: %s\n", i + 1, cmd->args[i]);
        }
    }

void executeCommand(Command *cmd)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("\nChild PID: %d\n", getpid());

        execvp(cmd->args[0], cmd->args);

        perror("execvp failed");
        exit(1);
    }
    else
    {
        waitpid(pid, NULL, 0);

        printf("\nCommand execution completed.\n");
    }
}

void freeCommand(Command *cmd)
{
    for (int i = 0; i < cmd->count; i++)
        free(cmd->args[i]);
}

```

```

        free(cmd->args[1]);
    }

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("\nmyShell> ");
        fflush(stdout);

        int n = read(STDIN_FILENO, input, MAX_INPUT - 1);

        if (n <= 0)
            break;

        input[n] = '\0';

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        if (strcmp(input, "exit") == 0)
            break;

        Command cmd;

        if (!parseCommand(input, &cmd))
            continue;

        if (cmd.count == 0)

```

```

        if (cmd.count == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        displayParsing(&cmd);

        executeCommand(&cmd);

        freeCommand(&cmd);
    }

    printf("\nShell terminated.\n");

    return 0;
}

```

OUTPUT:

```

[varshithcomple@VARSHITHs-MacBook-Air ~ % nano taskk2.c
[varshithcomple@VARSHITHs-MacBook-Air ~ % gcc taskk2.c -otaskk2
clang: warning: overriding deployment version from '16.0' to '26.0' [-Woverriding-deployment-version]
[varshithcomple@VARSHITHs-MacBook-Air ~ % ./taskk2

myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Child PID: 96022
Hello

Command execution completed.

myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Child PID: 96128
Hello World

Command execution completed.

myShell> █

```

TASK 3:


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 20
#define MAX_LEN 1024

int main() {
    char input[MAX_LEN];

    while (1) {

        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        /* Check unmatched double quote */
        int quoteCount = 0;
        for (int i = 0; input[i]; i++) {
            if (input[i] == '"')
                quoteCount++;
        }

        if (quoteCount % 2 != 0) {
            printf("Syntax Error: Unmatched double quote.\n");
            continue;
        }

        char *args[MAX_ARGS];
        int argc = 0;

        char parsed[MAX_LEN];
        int p = 0;

        for (int i = 0; input[i]; ) {

            while (input[i] == ' ')
                i++;

            if (input[i] == '\0')
                break;

            char token[256];
            int t = 0;

            /* Handle double quoted string */
            if (input[i] == '"') {
                i++;

                while (input[i] && input[i] != '"') {

                    if (input[i] == '$') {

                        i++;

                        char var[100];
                        int v = 0;

                        while ((input[i] >= 'A' && input[i] <= 'Z') ||
                               (input[i] >= 'a' && input[i] <= 'z') ||
                               input[i] == '_') {

                            var[v++] = input[i++];
                        }

                        var[v] = '\0';

                        char *value = getenv(var);

                        if (value)

```

```

        if (value)
            for (int k = 0; value[k]; k++)
                token[t++] = value[k];
        }
        else {
            token[t++] = input[i++];
        }
    }

    i++;
}
else {
    while (input[i] && input[i] != ' ')
        token[t++] = input[i++];
}

token[t] = '\0';

args[argc] = malloc(strlen(token) + 1);
strcpy(args[argc], token);
argc++;
}

args[argc] = NULL;

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++) {
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid == 0) {

    printf("\nChild Process\n");
    printf("PID = %d\n\n", getpid());

    execvp(args[0], args);

```

```

    perror("execvp");
    exit(1);
}
else {

    waitpid(pid, NULL, 0);

    printf("\nChild process completed.\n");
}

for (int i = 0; i < argc; i++)
    free(args[i]);
}

return 0;
}

```

OUTPUT:

```
[varshithcomple@VARSHITHS-MacBook-Air ~ % nano taskk3.c
[varshithcomple@VARSHITHS-MacBook-Air ~ % gcc taskk3.c -otaskk3
clang: warning: overriding deployment version from '16.0' to '26.0' [-Woverriding-deployment-version]
[varshithcomple@VARSHITHS-MacBook-Air ~ % ./taskk3
myShell> echo 'Hello $user'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: 'Hello
Argument 3: $user'

Child Process
PID = 98594

'Hello $user'

Child process completed.
myShell> █
```